AI Fitness Coach Dispatch — End-to-End Guide

Serverless AI coach on AWS: Bedrock (Claude) + Lambda + API Gateway + DynamoDB + EventBridge dispatch + SNS

IAM · Bedrock · Lambda · API Gateway · DynamoDB · Scheduler · SNS

AI Fitness Coach Dispatch — End-to-End Project Guide

Build a serverless **AI fitness coach** on AWS. Users submit their profile and goals; **Claude (via Amazon Bedrock)** generates a personalized workout & nutrition plan; an event-driven **"dispatch"** delivers coaching messages to users on demand and on a daily schedule.

> Based on the NextWork project *"Build a Fitness Coach with Claude"* (AI Fitness Coach Dispatch).

> The page is an interactive app, so this guide is a complete, buildable reconstruction of the

> project's intent — cross-check it against the live NextWork steps as you go.

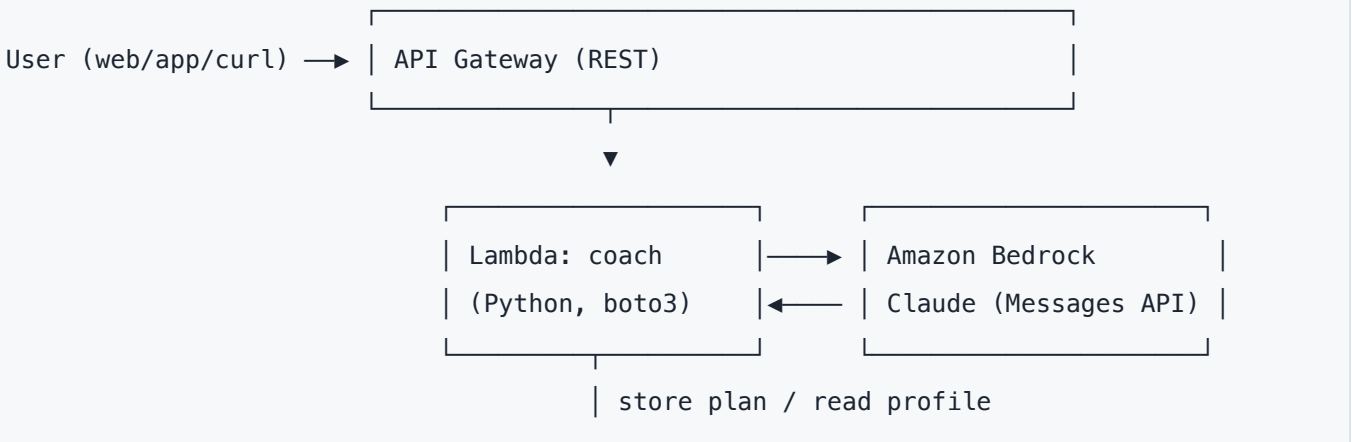
---

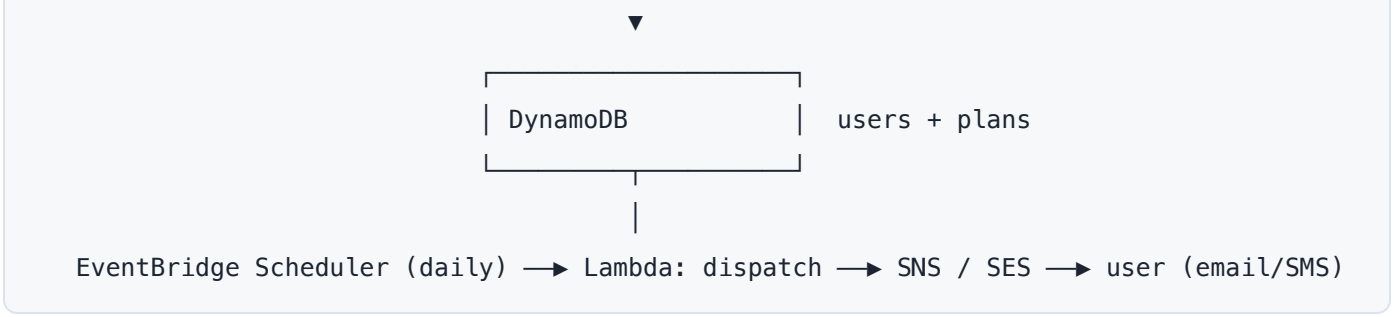
1. What you're building

A user sends their details (age, weight, goal, fitness level, constraints). The system asks **Claude on Bedrock** to write a tailored plan, stores it, and **dispatches** it back to the user — two ways:

- **On-demand**: an HTTP request (API Gateway → Lambda → Bedrock) returns a plan immediately.
- **Scheduled dispatch**: EventBridge Scheduler fires daily → Lambda generates today's coaching message per user → **SNS/SES** emails/texts it.

Architecture





Services: IAM · Amazon Bedrock (Claude) · Lambda (Python) · API Gateway · DynamoDB · EventBridge Scheduler · SNS (or SES) · CloudWatch Logs · (optional) Secrets Manager.

Why serverless? Nothing runs when idle — you pay per request + per Bedrock token. No servers to patch. Scales automatically.

---

2. Prerequisites

- AWS account + AWS CLI configured (`aws configure`), Python 3.12 locally.
- **Enable Bedrock model access** (one-time): Bedrock console → **Model access** → request access to the **Anthropic Claude** models → wait until *Access granted*. Nothing works until this is done.
- Pick your region (e.g. `us-east-1`) and confirm Claude is available there.
- Note your **Bedrock model ID**. On Bedrock, Anthropic IDs take an `anthropic.` prefix, and current Claude models are usually invoked through a **cross-region inference profile** — e.g. `us.anthropic.claude-...`. Get the exact ID from Bedrock console → *Model access* / *Model catalog* (or `aws bedrock list-inference-profiles`). This guide uses `MODEL_ID` as a placeholder.

---

3. Step 1 — IAM (least privilege)

Create an execution **role** for the Lambdas (not a user). Trust policy = Lambda; attach a policy granting exactly what's needed.

`trust-policy.json`

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
```

```
    "Principal": { "Service": "lambda.amazonaws.com" },
    "Action": "sts:AssumeRole" }] }
```

`coach-policy.json` (least privilege)

```
{ "Version": "2012-10-17", "Statement": [
  { "Sid": "Bedrock", "Effect": "Allow",
    "Action": ["bedrock:InvokeModel"], "Resource": "*" },
  { "Sid": "DynamoDB", "Effect": "Allow",
    "Action": ["dynamodb:GetItem","dynamodb:PutItem","dynamodb:Query","dynamodb:Scan"],
    "Resource": "arn:aws:dynamodb:*:*:table/FitnessCoach*" },
  { "Sid": "Notify", "Effect": "Allow",
    "Action": ["sns:Publish"], "Resource": "*" },
  { "Sid": "Logs", "Effect": "Allow",
    "Action": ["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],
    "Resource": "*" }
]}
```

```
aws iam create-role --role-name FitnessCoachRole \
  --assume-role-policy-document file://trust-policy.json
aws iam put-role-policy --role-name FitnessCoachRole \
  --policy-name FitnessCoachPolicy --policy-document file://coach-policy.json
```

> `bedrock:InvokeModel` with `Resource:"*"` is fine for learning; tighten to the model ARN later.

---

4. Step 2 — DynamoDB (data store)

Two tables (or one with a sort key). Simple version — one table keyed by `userId`:

```
aws dynamodb create-table --table-name FitnessCoachUsers \
  --attribute-definitions AttributeName=userId,AttributeType=S \
  --key-schema AttributeName=userId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

Item shape:

```
{ "userId": "u123", "name": "Pramod", "email": "you@example.com",
  "age": 34, "weightKg": 78, "goal": "lose fat, build strength",
  "level": "intermediate", "constraints": "bad left knee, 4 days/week",
  "lastPlan": "…generated plan…", "updatedAt": "2026-08-12T09:00:00Z" }
```

Seed one user:

```
aws dynamodb put-item --table-name FitnessCoachUsers --item '{
  "userId":{"S":"u123"},"name":{"S":"Pramod"},"email":{"S":"you@example.com"},
  "age":{"N":"34"},"weightKg":{"N":"78"},"goal":{"S":"lose fat, build strength"},
  "level":{"S":"intermediate"},"constraints":{"S":"bad left knee, 4 days/week"}}'
```

---

5. Step 3 — The coach Lambda (Claude on Bedrock)

This is the heart of the project. It builds a **prompt** from the user's profile and calls Claude through Bedrock using the **Messages API** body shape. Lambda already ships `boto3`, so **no extra dependencies** are needed.

`lambda_coach.py`

```
import json, os, boto3
from datetime import datetime, timezone

bedrock  = boto3.client("bedrock-runtime")
ddb      = boto3.resource("dynamodb")
table    = ddb.Table(os.environ.get("USERS_TABLE", "FitnessCoachUsers"))
MODEL_ID = os.environ["MODEL_ID"]   # e.g. us.anthropic.claude-... (from Bedrock console)

SYSTEM = (
    "You are an expert, encouraging personal fitness coach. "
    "Given a user's profile, produce a safe, specific, actionable plan. "
    "Respect injuries and time constraints. Never give medical advice; "
    "add a one-line disclaimer to consult a doctor before starting."
)

def build_prompt(u):
    return (
```

```
        f"Create a personalized weekly fitness and nutrition plan.\n\n"
        f"Name: {u.get('name')}\nAge: {u.get('age')}\nWeight: {u.get('weightKg')} kg\n"
        f"Fitness level: {u.get('level')}\nGoal: {u.get('goal')}\n"
        f"Constraints: {u.get('constraints')}\n\n"
        f"Return: (1) a 4-week overview, (2) a day-by-day week-1 workout split, "
        f"(3) simple nutrition guidance, (4) one motivational tip. Use clear headings."
    )

def ask_claude(prompt):
    body = {
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 1500,
        "system": SYSTEM,
        "messages": [{"role": "user", "content": prompt}],
    }
    resp = bedrock.invoke_model(modelId=MODEL_ID, body=json.dumps(body))
    payload = json.loads(resp["body"].read())
    # Claude Messages response: content is a list of blocks; take the text blocks
    return "".join(b.get("text", "") for b in payload.get("content", []) if b.get("type")
== "text")

def handler(event, context):
    # API Gateway proxy: body is a JSON string. Also supports direct invoke.
    body = json.loads(event["body"]) if isinstance(event.get("body"), str) else event
    user_id = body.get("userId", "u123")

    item = table.get_item(Key={"userId": user_id}).get("Item")
    if not item:
        return {"statusCode": 404, "body": json.dumps({"error": "user not found"})}

    plan = ask_claude(build_prompt(item))

    table.update_item(
        Key={"userId": user_id},
        UpdateExpression="SET lastPlan = :p, updatedAt = :t",
        ExpressionAttributeValues={":p": plan, ":t":
datetime.now(timezone.utc).isoformat()},
    )
    return {"statusCode": 200,
```

```
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"userId": user_id, "plan": plan})}
```

Deploy it:

```
zip coach.zip lambda_coach.py
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
aws lambda create-function --function-name FitnessCoach \
  --runtime python3.12 --handler lambda_coach.handler \
  --role arn:aws:iam::$ACCOUNT:role/FitnessCoachRole \
  --timeout 60 --memory-size 256 --zip-file fileb://coach.zip \
  --environment "Variables=
{USERS_TABLE=FitnessCoachUsers,MODEL_ID=REPLACE_WITH_YOUR_MODEL_ID}"
```

Test it directly:

```
aws lambda invoke --function-name FitnessCoach \
  --payload '{"userId":"u123"}' --cli-binary-format raw-in-base64-out out.json && cat
out.json
```

> **Prefer the Anthropic SDK?** Package the `anthropic` library in a Lambda layer and use

> `AnthropicBedrockMantle(aws_region="us-east-1")` → `client.messages.create(model="anthropic.claude-opus-4-8", max_tokens=1500, messages=[...])`.

> boto3 `invoke_model` (above) needs no layer, which is why it's the default here.

---

6. Step 4 — API Gateway (on-demand path)

Expose the coach over HTTP so a web/app/curl client can request a plan.

```
# HTTP API is the quickest; REST API also works.
API_ID=$(aws apigatewayv2 create-api --name FitnessCoachAPI --protocol-type HTTP \
  --target arn:aws:lambda:us-east-1:$ACCOUNT:function:FitnessCoach --query ApiId --output
text)

aws lambda add-permission --function-name FitnessCoach \
```

```
  --statement-id apigw --action lambda:InvokeFunction \
  --principal apigateway.amazonaws.com \
  --source-arn "arn:aws:execute-api:us-east-1:$ACCOUNT:$API_ID/*/*"

echo "Endpoint: https://$API_ID.execute-api.us-east-1.amazonaws.com"
```

Call it:

```
curl -s -X POST https://$API_ID.execute-api.us-east-1.amazonaws.com \
  -H 'Content-Type: application/json' -d '{"userId":"u123"}' | jq -r .plan
```

---

7. Step 5 — The dispatch (scheduled delivery)

This is the "**dispatch**" in the project name: a scheduled job that generates and **sends** today's coaching to users automatically.

`lambda_dispatch.py`

```
import os, json, boto3

lam  = boto3.client("lambda")
sns  = boto3.client("sns")
ddb  = boto3.resource("dynamodb")
table = ddb.Table(os.environ.get("USERS_TABLE", "FitnessCoachUsers"))
TOPIC = os.environ["TOPIC_ARN"]

def handler(event, context):
    users = table.scan().get("Items", [])
    sent = 0
    for u in users:
        # reuse the coach lambda to generate a fresh message
        resp = lam.invoke(FunctionName="FitnessCoach",
                          Payload=json.dumps({"userId": u["userId"]}))
        plan = json.loads(resp["Payload"].read()).get("plan", "")
        sns.publish(TopicArn=TOPIC,
                    Subject=f"Your coaching for today, {u.get('name','')}",
                    Message=plan[:2000])   # SMS/email friendly
```

```
        sent += 1
    return {"sent": sent}
```

Set up the topic + subscription + schedule:

```
TOPIC_ARN=$(aws sns create-topic --name FitnessCoachDispatch --query TopicArn --output
text)
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint
you@example.com
# Confirm the email subscription from your inbox.

zip dispatch.zip lambda_dispatch.py
aws lambda create-function --function-name FitnessCoachDispatch \
  --runtime python3.12 --handler lambda_dispatch.handler \
  --role arn:aws:iam::$ACCOUNT:role/FitnessCoachRole \
  --timeout 120 --memory-size 256 --zip-file fileb://dispatch.zip \
  --environment "Variables={USERS_TABLE=FitnessCoachUsers,TOPIC_ARN=$TOPIC_ARN}"

# Daily at 07:00 UTC via EventBridge Scheduler
aws scheduler create-schedule --name daily-coach-dispatch \
  --schedule-expression "cron(0 7 * * ? *)" \
  --flexible-time-window '{"Mode":"OFF"}' \
  --target "{\"Arn\":\"arn:aws:lambda:us-east-
1:$ACCOUNT:function:FitnessCoachDispatch\",\"RoleArn\":\"arn:aws:iam::$ACCOUNT:role/Fitne
ssCoachRole\"}"
```

> Add `scheduler.amazonaws.com` to the role trust policy (or use a dedicated scheduler role) and

> grant the role `lambda:InvokeFunction` on both functions.

---

8. Step 6 — Test end to end

1. **On-demand:** `curl` the API → you get a plan JSON, and the item's `lastPlan` updates in DynamoDB.
2. **Scheduled:** manually invoke `FitnessCoachDispatch` → confirm the email arrives via SNS.
3. **Logs:** watch **CloudWatch Logs** for each function; check Bedrock latency and token usage.

```
aws lambda invoke --function-name FitnessCoachDispatch --payload '{}' \
  --cli-binary-format raw-in-base64-out d.json && cat d.json
```

9. Common issues & fixes (interview-ready)

Symptom	Cause	Fix
`AccessDeniedException` on invoke	Model access not granted / wrong model ID	Grant access in Bedrock → *Model access*; use the exact inference-profile ID
`ValidationException` body	Wrong request shape	Must include `anthropic_version:"bedrock-2023-05-31"` + `messages` + `max_tokens`
Lambda timeout	Bedrock call > timeout	Raise Lambda timeout (60–120s); keep `max_tokens` reasonable
Empty `plan`	Parsed wrong field	Response `content` is a **list of blocks**; concat the `text` blocks
403 from API Gateway	Missing `lambda:InvokeFunction` permission	Add the `add-permission` statement
Dispatch sends nothing	Email subscription unconfirmed	Confirm the SNS subscription link
Throttling / 429	Bedrock rate limits	Retry with exponential backoff; batch users

10. Why this design is good (for the write-up / interview)

- **Serverless & event-driven** — no idle cost, auto-scales, decoupled dispatch.
- **Least-privilege IAM** — a role scoped to Bedrock + one DynamoDB table + SNS + logs.
- **Separation of concerns** — `coach` generates, `dispatch` delivers, DynamoDB persists.
- **Prompt engineering** — a clear system prompt (safety, constraints, format) + a structured user prompt built from profile data; a disclaimer keeps it responsible.
- **Observability** — CloudWatch logs + Bedrock token metrics.
- **Extensible** — add SES for rich email, Step Functions for multi-step coaching, a feedback loop that stores user responses and adapts the next plan, or QuickSight for progress dashboards.

11. Cleanup (avoid charges)

```
aws scheduler delete-schedule --name daily-coach-dispatch
aws lambda delete-function --function-name FitnessCoach
aws lambda delete-function --function-name FitnessCoachDispatch
aws sns delete-topic --topic-arn $TOPIC_ARN
aws dynamodb delete-table --table-name FitnessCoachUsers
aws apigatewayv2 delete-api --api-id $API_ID
# then delete the IAM role/policy
```

Stretch goals

- **SES** for branded HTML emails; **SMS** via SNS for nudges.
- **Feedback loop**: store "did you complete it?" → feed into the next prompt (adaptive coaching).
- **Step Functions**: extract goals → generate plan → validate → dispatch, with retries.
- **Bedrock Guardrails** for safety; **prompt caching** for the system prompt to cut cost.
- **IaC**: capture all of the above in Terraform or AWS SAM for one-command deploys.